

INF1103: Programming Fundamentals with DevOps

Week 2: Control Structures and Flow Control



Outline

- Understanding Control Structures
- Conditions and Operators
- Loops for Repetition

Can Programs make decisions?

How does Netflix recommend movies?

- A. Humans manually control every decision
- B. Computers follow a set of rules
- C. Computers make random choices
- D. Magic ✨



Multiple Choice

What is Control flow?

- Conditional logic allows software to make decisions based on specific conditions or rules.



If you like sports content, show more sports videos.

Otherwise, recommend other content.



If a product is in stock, allow purchase.

Otherwise, display "Out of Stock."



If player health reaches 0, display "Game Over."

If the player collects enough points, unlock the next level.

What is Control Flow?

Order in which a program executes its statements and instructions.
It determines **which** code runs, **when** it runs, and under **what** **conditions**.



Control Flow Structures

1. Sequential Control Flow
2. Selection (Conditional) Control Flow
3. Iteration (Looping) Control Flow



Control Flow Structures

- 1. Sequential Control Flow**
2. Selection (Conditional) Control Flow
3. Iteration (Looping) Control Flow

Sequential Control Flow



Sequential Control Flow

- Statements are executed one after another in the order they appear.

```
first = 15
second = 50
print(first, second)
first, second = second, first
print(first, second)
```

Execution order: Line 1 → Line 2 → Line 3 → Line 4 → Line 5

```
15 50
50 15
```

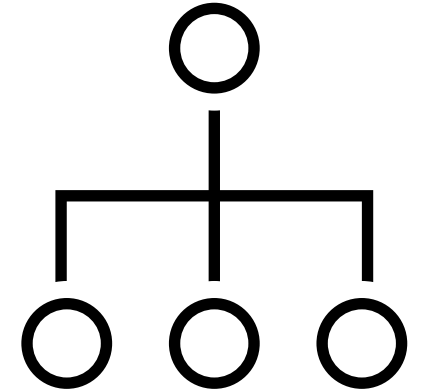


Control Flow Structures

1. Sequential Control Flow
2. **Selection (Conditional) Control Flow**
3. Iteration (Looping) Control Flow

Conditional Control Flow

The program makes decisions based on the **truth value of conditions**, determining which path to take.



Conditional Control Flow

- The program makes decisions based on the **truth value of conditions**, determining which path to take.
- Boolean variable type
 - True or False
- if-else statement
 - if, if-else, if-elif-else
- Boolean expression
 - An expression that evaluates to either True or False.

Conditional Control Flow

- True or False
 - *bool* is a subclass of *int*
 - `True == 1`
 - `False == 0`
 - `True + True = 2`

Conditional Control Flow: if statement

- The **if statement** allows programs to make decisions based on conditions that evaluate to True or False.
- Conditions inside the if statement determine which block of code executes depending on their True or False value.

Conditional Control Flow: if statement

if **expr**:

statement(s)

```
food = "Burger"  
price = 0.0  
if food == "Burger":  
    print("you chose burger")  
    print("please pay 8.00 SGD")  
    price = 8.0
```

```
you chose burger  
please pay 8.00 SGD
```

Conditional Control Flow

if **expr:**
statement(s)

1. Comparisons

- if `x == 5`:
- if `a < b`:
- Always return True or False
- *if x=5: → error*

2. Numbers

- if `0`: → False
- if `10`: → True

3. Containers

- if `[]`: → False (empty list)
- if `[1, 2, 3]`: → True (non-empty)
- Works the same for strings, dicts, sets

4. None check

- Use explicit check

Conditional Control Flow: elif statement

```
if expr1:  
    statement(s)  
elif expr2:  
    statement(s)  
else:  
    statement(s)
```

Details of if statement

- The Colon (:)
 - The colon is required at the end of the if and else lines.
 - It indicates that a block of code will follow.
- Indentation:
 - Python uses indentation to define the scope of the code block.
 - Consistent indentation is crucial for the correct execution of the program.
 - Typically, **four spaces** or one tab is used for indentation.

Conditional Control Flow: elif statement

if **expr1**:
 statement(s)

elif **expr2**:
 statement(s)

else:
 statement(s)

```
food = "Sandwich"
price = 0.0
if food == "Fries":
    print("you chose Fries")
    print("please pay 4.00 SGD")
    price = 4.0
elif food == "Burger":
    print("you chose Burger")
    print("please pay 8.00 SGD")
    price = 8.0
else:
    print("sorry your choice not avialble")
```

Conditional Expression

- *value_if_true* if condition else *value_if_false*
- Is useful when you have a condition and you want to produce (return) a value based on that condition.



Control Flow Structures

1. Sequential Control Flow
2. Selection (Conditional) Control Flow
3. **Iteration (Looping) Control Flow**



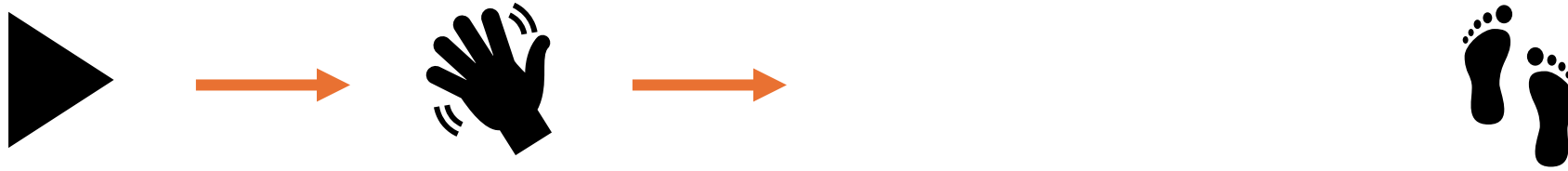
Iteration (Loop) Control

The program **repeatedly** executes a block of code as long as a **specified condition is met**.

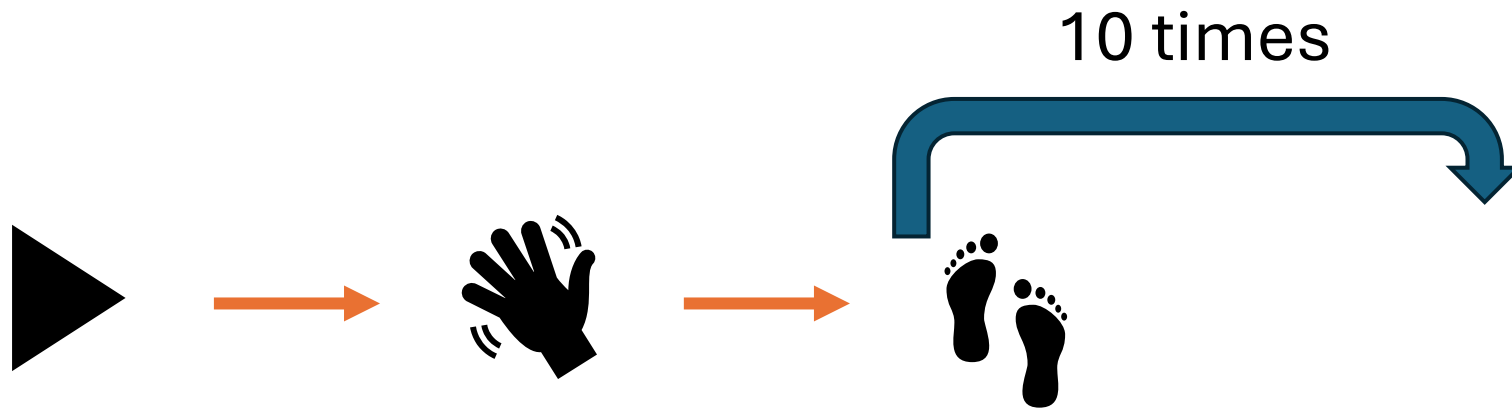
Iteration (Loop) Control



Iteration (Loop) Control



Iteration (Loop) Control



Iteration (Loop) Control: while

- Sequential Control

- Start
- Hello
- Walk Circle 1
- Walk Circle 2
- ...
- Walk Circle 10
- End

- Loop

- Pseudocode:

- Walk circle = 0
- IF Walk circle < 10
- Walk circle
- Increase Walk circle by 1
- ELSE
- Stop
- IF Walk circle < 10
- Walk circle
- Increase Walk circle by 1
- ELSE
- Stop
- ...

Iteration (Loop) Control: while

while expression:

statement1

statement2

```
walk_circle = 0
while walk_circle < 10:
    walk_circle = walk_circle + 1
    print("walking complete")
    print(walk_circle)
```

- Expression
 - Boolean Expression
 - Whether to stay within the loop
- Statements
 - Do something
 - Will change the result of the Expression

Iteration (Loop) Control: infinite loops

- What happens if you forget to increment `walk_circle`?
 - Dead Loop
 - The most common reason for a program 'hanging'
 - To quit program execution: restart, Ctrl + C
- When Infinite Loop is useful?
while True:
 - Continuous input loop → keep asking user until a quit command is given
 - Server or event listener → wait for connections or events indefinitely
 - Game loop → constantly update and render until game is over

Iteration (Loop) Control: for

provides the ability to **loop over the items of any sequence,**
such as a list, tuple or a string.

Iteration (Loop) Control: for

for iterating_var **in** sequence:
 statements

- Define walk_circle inside **for**
- **Sequence**
 - range(10)

```
walk_circle = 0
for walk_circle in range(10):
    print(walk_circle)
print("walking complete")
print(walk_circle)
```

range()

- Built-in function returning a range object, not a list
- range(stop)
- range(start, stop)
- range(start, stop, step)
 - Start: starting num of the sequence
 - Stop: generate num up to, but **not including this num**
 - Step: difference between each num in the sequence

Loop while vs for

while boolean_expression:
 Statements

- Condition met or not

for iterating_var **in** sequence:
 Statements

- Go through the sequence
- List, tuple, string, range
- Iterable:
 - Whenever we say an iterable in Python, it means a sequence data type (for example, a list).

Iteration (Loop) Control

- What will be the condition that loop may not be executed?
- What is a dead loop?
 - loop over the items of a sequence

Break Statement

- Stop the loop when some condition is satisfied
- Can be used in **both Python while and for loops**

```
walk_circle = 0
for walk_circle in range(1,10):
    print(walk_circle)
    if walk_circle == 5:
        break
print("walking complete")
print(walk_circle)
```

Continue Statement

- Skips the remaining statements in the current loop and starts the next iteration.
- Can be used in **both Python while and for loops**

```
walk_circle = 0
for walk_circle in range(1,10):
    if walk_circle == 5:
        continue
    print(walk_circle)
print("walking complete")
print(walk_circle)
```

Task for the day!!

- Traffic Light Simulator
 - Rules:
 - If the light is Green → Print "Go"
 - If the light is Yellow → Print "Slow Down"
 - If the light is Red → Print "Stop"
 - The simulation should run for 10 seconds.



Takeaway!!

- Logic defines the decision-making process in a program.
- Commonly implemented using If-Then and If-Then-Else blocks.
- Control flows manages how a program moves from one block to another. (Sequence, conditional or iteration)

Logic decides what to do; Control Flow decides when and in what order to do it.

Thanks!!!

